```
(-elem-index 7 '(3 4 5 6 7 8))
4
```

If you want to find all the indices in which a given element is located, you can use the '-elem-indices' function. In the following example, the locations at which the element '1' is present in the list are returned.

```
(-elem-indices element list) ;; Syntax

(-elem-indices 1 '(0 1 2 0 1 2 3 4 5))
(1 4)
```

The '-find-index' function takes a predicate function and an input list, and returns the index of the element that first matches the predicate. For example:

```
(-find-index predicate list) ;; Syntax


(-find-index 'even? '(1 2 3 4 5))
1
```

The '-find-last-index' function also takes a predicate function and an input list, and returns the last element in the list that matches the predicate function. In the following example, the element '6' satisfies the predicate function 'even?', and is located in position 4.

```
(-find-last-index predicate list) ;;Syntax


(-find-last-index 'even? '(2 3 4 5 6 7))
4
```

The '-find-indices' function returns all the indices whose elements match the predicate function. An example is given below:

```
(-find-indices predicate list) ;; Syntax

(-find-indices 'even? '(2 3 4 5 6 7))
(0 2 4)
```

Set operations

The '-union' function returns a set union of the input lists passed as arguments. For example:

```
(-union list1 list2) ;; Syntax

(-union '(1 2) '(3 4))
(1 2 3 4)
```

The '-difference' function returns the elements of list1 that are not in list2. An example is given below:

```
(-difference list1 list2) ;; Syntax


(-difference '(1 2) '(3 4))
(1 2)
```

If you want to find the elements that exist in both the input lists, then you can use the '-intersection' function as follows:

```
(-intersection list1 list2) ;; Syntax

(-intersection '(1 2 3) '(3 4 5))
(3)
```

The '-power-set' function returns the power set of the input list. A power set is the set of all subsets of a set, including the empty set and the set itself. An example is shown below:

```
(-powerset list) ;; Syntax


(-powerset '(1 2))
((1 2) (1) (2) nil)
```

You can generate all the permutations of the input elements in a set using the '-permutations' function as illustrated below:

```
(-permutations list) ;; Syntax


(-permutations '(a b c))
((a b c) (a c b) (b a c) (b c a) (c a b) (c b a))
```

The '-distinct' function returns a list with no duplicate elements. For example:

```
(-distinct list) ;; Syntax


(-distinct '(1 2 3 4 4 5))
(1 2 3 4 5)
```

We will explore more functions from dash.el in the next article. END

By: Shakthi Kannan

The author is a free software enthusiast and blogs at *shakthimaan.com*.